STAT 185 - Problem Set #5

Matt Krasnow

Instructions

This assignment covers nonlinear manifold learning (ISOMAP and LLE) through a mixture of theory and coding. You will need the file *pset5.Rdata* available on Canvas. The commands for loading the data assume this file, *pset5.Rmd*, and the Rdata file are in the same folder. Unfortunately, there is not one R package that provides each of the dimension reduction methods we've discussed in class.

We will be using the **Rdimtools** package to implement some of the methods, which has a general form **out <- do.method(X,ndim = d, type = c("knn",k))**. The output dimension is specified by the variable d, the number of nearest neighbors is k, and the lower dimensional representation is contain in out$Y. For ISOMAP, use do.iosmap and for Laplacian Eigenmaps, use do.lapeig. For Laplacian eigenmaps, you can specify the bandwidth σ^2 by adjusting **kernelscale** in the command for LEs.

For Hessian Local Linear Embeddings, we'll use the **dimRed** package, which requires the command **out<-embed(X,ndim = d, "HLLE", knn = k)** to use k nearest neighbors Complete the following questions and submit your solutions as a *pdf* via Canvas by 11:59 PM on Friday November 14th.

Problems

Problem 0

Please indicate your topic for the final report and oral exam.

Manifold learning for High-Dimensional Energy Optimization. I want to investigate the methods and do a short investigation on the IRED energy trajectories from https://energy-based-model.github.io/ired/

Problem 1

The variable *faces* contains 33 images (112×92 pixels) of a person taken at different angles. The data were obtained (and modified slightly) from the github repository maintained by Yuan Yao. Three images are shown below.

```
par(pin = c(1.3,1), mfrow = c(1,3))
for (j in c(1,30,25)){
  plot_face(faces[j,])
}
```





Each image is a function of the angle of the camera used to take the picture so it is natural to expect the data reside on a one dimensional manifold. Let's investigate that assumption. For each sample, $\vec{x}_i$, compute

its five nearest neighbors $\vec{x}_{i_1}, \ldots, \vec{x}_{i_5}$. Then compute and save the nonzero singular values of

$$\mathbf{M}_i = \begin{bmatrix} \vec{x}_{i_1}^T - \vec{x}_i^T \\ \vdots \\ \vec{x}_{i_5}^T - \vec{x}_{i_1}^T \end{bmatrix}.$$

a. Generate boxplots of the singular values. Do these results suggests (or at least agree with the assumption) that the data reside on a one-dimensional manifold?

```r
k <- 5
n <- nrow(faces)

knn_res <- KernelKnn::knn.index.dist(as.matrix(faces), k = k)
nn_idx  <- knn_res$train_knn_idx   # n x k matrix of neighbor indices
singvals <- matrix(NA_real_, nrow = n, ncol = k)

for (i in 1:n) {
  x_i   <- faces[i, ]
  neigh <- faces[nn_idx[i, ], , drop = FALSE]

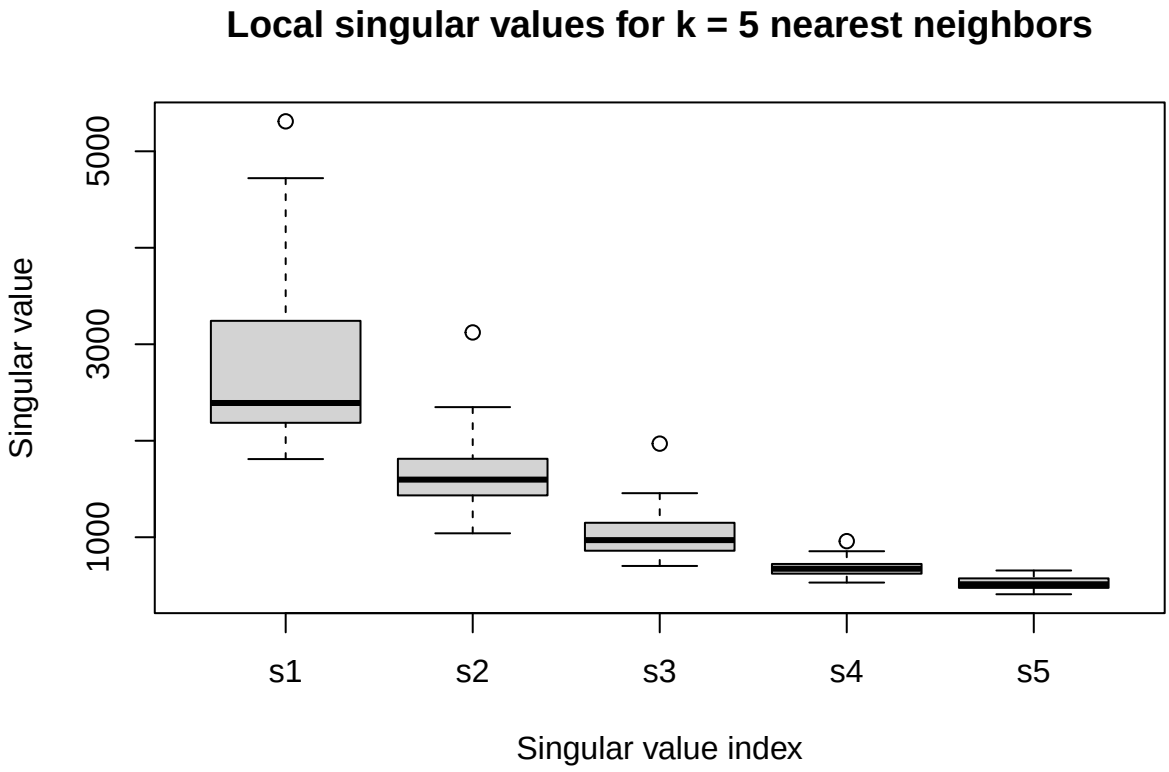
  M_i <- sweep(neigh, 2, x_i, FUN = "-")

  s <- svd(M_i)$d

  # store up to k singular values
  singvals[i, 1:min(length(s), k)] <- s[1:min(length(s), k)]
}

colnames(singvals) <- paste0("s", 1:k)

boxplot(as.data.frame(singvals),
        xlab = "Singular value index",
        ylab = "Singular value",
        main = "Local singular values for k = 5 nearest neighbors")
```



No, these results do NOT agree that the data lies on a 1D manifold. The second singular value is almost as large as the first one and there is overlap in their ranges. It is likely that the other ones are important.

b. If you use $k = 10$ or $k = 20$ nearest neighbors, do your results change? What is a potential reason for these results?

```
k <- 20
n <- nrow(faces)

knn_res <- KernelKnn::knn.index.dist(as.matrix(faces), k = k)
nn_idx  <- knn_res$train_knn_idx   # n x k matrix of neighbor indices
singvals <- matrix(NA_real_, nrow = n, ncol = k)

for (i in 1:n) {
  x_i   <- faces[i, ]
  neigh <- faces[nn_idx[i, ], , drop = FALSE]

  M_i <- sweep(neigh, 2, x_i, FUN = "-")

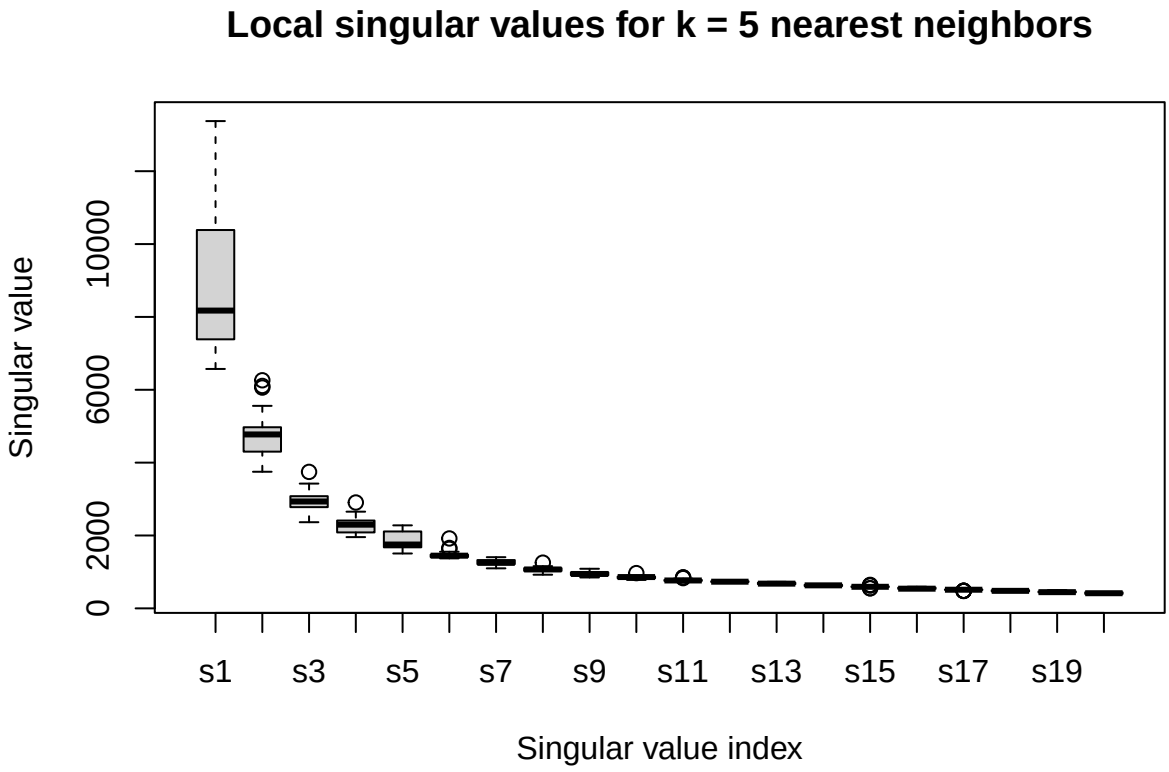
  s <- svd(M_i)$d

  # store up to k singular values
  singvals[i, 1:min(length(s), k)] <- s[1:min(length(s), k)]
}

colnames(singvals) <- paste0("s", 1:k)

boxplot(as.data.frame(singvals),
```

```
        xlab = "Singular value index",
        ylab = "Singular value",
        main = "Local singular values for k = 5 nearest neighbors")
```



Yes, with k=20, it looks like the data is well represented by a 1D manifold. The lowest point in the first singular value boxplot is larger than the largest value in the next singular value box plot. While this isn't decisive, it shows that it is more likely to be on a 1D manifold when we use k=20

Problem 2

However, choosing a manifold learning method for the *faces* data is less clear. In this problem, we'll choose among HLLEs, Laplacian Eigenmaps (LEs), and ISOMAP while the tuning of the number of nearest neighbors.

The helper function **neighbor_match(X,Y,k)** takes in a high-dimensional data matrix X, a lower dimensional data matrix, Y, given by a dimension reduction algorithm, and a specified number of neighbors k and returns the average proportion of the k nearest neighbors preserved between X and Y. A neighbor_match of 0 is poor and 1 is good.

Plot the neighborhood match score for each method (HLLE, LE, ISOMAP) as a function of the number of nearest neighbors. For technical reasons, $k \geq 3$ for the algorithms to work. Use the default bandwidth for the rbf kernel in LEs. Based on these results, which method and value of k do you choose? Explain your reasoning.

```
k_vals   <- 3:10
methods  <- c("HLLE", "LE", "ISOMAP")
nm_scores <- matrix(NA_real_, nrow = length(k_vals), ncol = length(methods))
colnames(nm_scores) <- methods

faces_df      <- as.data.frame(faces)
faces_dimred  <- dimRedData(faces_df)
```

```
for (i in seq_along(k_vals)) {
  k <- k_vals[i]

  ## HLLE
  hlle_try <- try(
    embed(faces_dimred, "HLLE", knn = k, ndim = 2),
    silent = TRUE
  )

  if (!inherits(hlle_try, "try-error")) {
    Y_hlle <- as.matrix(hlle_try@data@data[, 1:2])
    nm_scores[i, "HLLE"] <- neighbor_match(faces, Y_hlle, k)
  } else {
    nm_scores[i, "HLLE"] <- NA_real_
  }

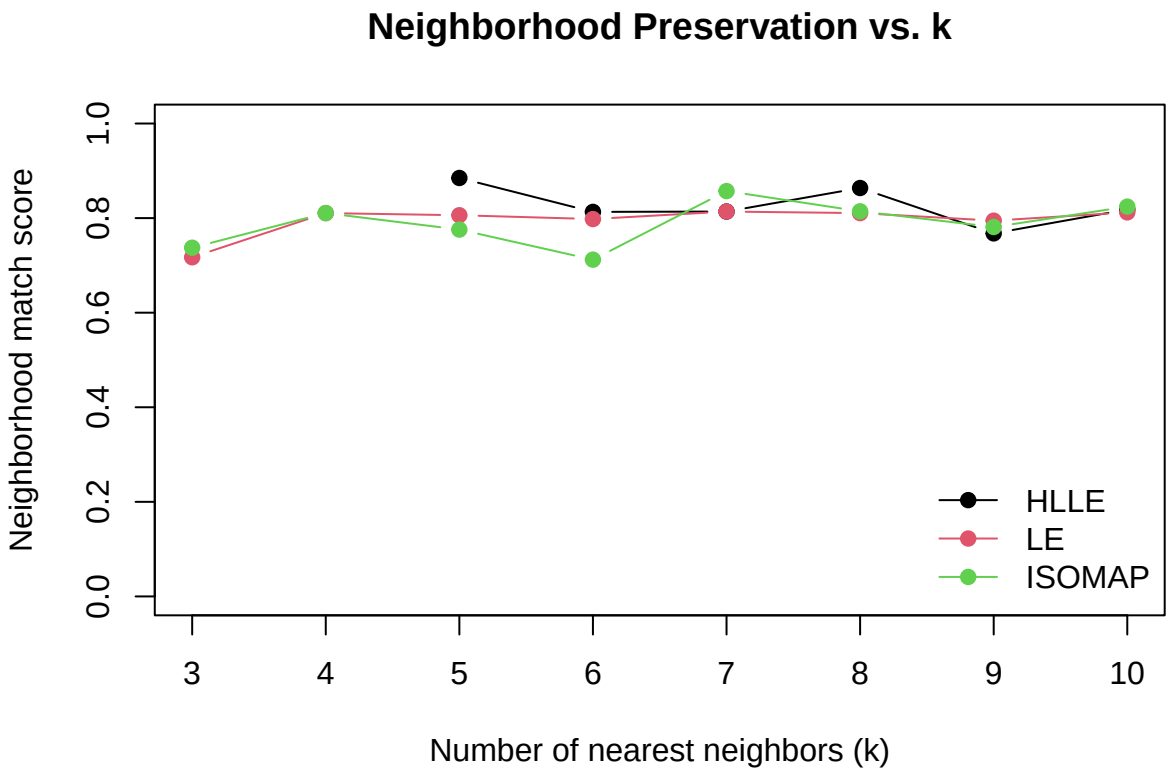
  ## Laplacian Eigenmap
  le_res <- do.lapeig(faces, ndim = 2, type = c("knn", k))
  Y_le   <- le_res$Y
  nm_scores[i, "LE"] <- neighbor_match(faces, Y_le, k)

  ## ISOMAP
  iso_res <- do.isomap(faces, ndim = 2, type = c("knn", k))
  Y_iso   <- iso_res$Y
  nm_scores[i, "ISOMAP"] <- neighbor_match(faces, Y_iso, k)
}
```

```
## 1/332/333/334/335/336/337/338/339/3310/3311/3312/3313/3314/3315/3316/3317/3318/3319/3320/3321/3322/33
## 1/332/333/334/335/336/337/338/339/3310/3311/3312/3313/3314/3315/3316/3317/3318/3319/3320/3321/3322/33
## 1/332/333/334/335/336/337/338/339/3310/3311/3312/3313/3314/3315/3316/3317/3318/3319/3320/3321/3322/33
## 1/332/333/334/335/336/337/338/339/3310/3311/3312/3313/3314/3315/3316/3317/3318/3319/3320/3321/3322/33
## 1/332/333/334/335/336/337/338/339/3310/3311/3312/3313/3314/3315/3316/3317/3318/3319/3320/3321/3322/33
## 1/332/333/334/335/336/337/338/339/3310/3311/3312/3313/3314/3315/3316/3317/3318/3319/3320/3321/3322/33
## 1/332/333/334/335/336/337/338/339/3310/3311/3312/3313/3314/3315/3316/3317/3318/3319/3320/3321/3322/33
## 1/332/333/334/335/336/337/338/339/3310/3311/3312/3313/3314/3315/3316/3317/3318/3319/3320/3321/3322/33
```

```
matplot(
  k_vals, nm_scores,
  type = "b", pch = 19, lty = 1,
  xlab = "Number of nearest neighbors (k)",
  ylab = "Neighborhood match score",
  ylim = c(0, 1),
  main = "Neighborhood Preservation vs. k"
)
legend(
  "bottomright",
  legend = methods,
  col = 1:ncol(nm_scores),
  lty = 1,
  pch = 19,
```

```
  bty = "n"
)
```



Based on the given information, we are trying to optimize for the model and k selection that maximizes the neighborhood match score. From the rresults, we can see that the highest score is associated with HLLE with k=5.

Problem 3

Using the optimal method and number of neighbors found in problem 2, plot the first 32 images (ordered from the lowest 1-dimensional coordinate to the largest 1-dimensional coordinate) in an 8×4 grid. What feature of the images is the selected manifold learning method capturing?

```
best_idx <- which(nm_scores == max(nm_scores, na.rm = TRUE), arr.ind = TRUE)
best_k <- k_vals[best_idx[1, "row"]]
best_method <- colnames(nm_scores)[best_idx[1, "col"]]

if (best_method == "HLLE") {
  Y <- as.matrix(embed(faces_dimred, "HLLE", knn = best_k, ndim = 2)@data@data[, 1:2])
  y1 <- as.numeric(prcomp(Y)$x[, 1])
} else if (best_method == "LE") {
  y1 <- as.numeric(do.lapeig(faces, ndim = 1, type = c("knn", best_k))$Y[, 1])
} else {
  y1 <- as.numeric(do.isomap(faces, ndim = 1, type = c("knn", best_k))$Y[, 1])
}
```

```
## 1/332/333/334/335/336/337/338/339/3310/3311/3312/3313/3314/3315/3316/3317/3318/3319/3320/3321/3322/33
```

```
ord <- order(y1)
par(mfrow = c(8, 4), mar = c(0, 0, 0, 0))
```

```
for (i in ord[1:32]) plot_face(faces[i, ])
```



The manifold learning algorithms are learning the direction that the person's face is oriented (inteded!). I came to this conclusion because you look down each column, you see differences in the position that the person is looking.

Problem 4

Consider the set $A = (0, 2\pi) \times (-3, 3)$ and the isometric manifold map

$$\Psi(x, y) = \begin{bmatrix} \text{sign}(\pi - x)(1 + \cos(x)) \\ \sin(x) \\ y \end{bmatrix}$$

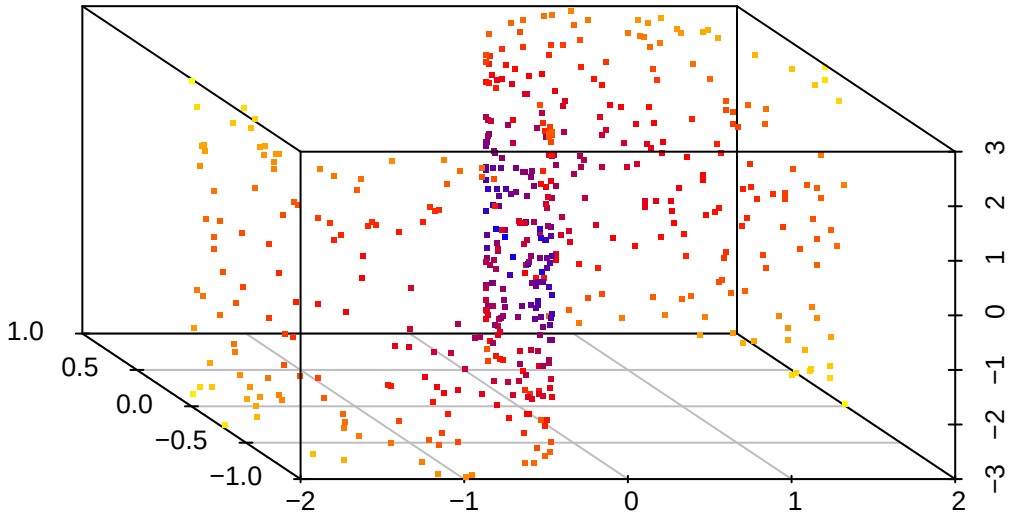
where $\text{sign}(z) = \begin{cases} 1 & z \geq 0 \\ -1 & \text{else.} \end{cases}$

Let

$$f(\vec{x}) = d^{\mathcal{M}}(\vec{x}, \vec{0})$$

be the geodesic distance between $\vec{x}$ and $\vec{0}$ along the manifold. Note, $\vec{0}$ is a point on the manifold so this is well-defined. Argue (mathematically or numerically) that $\|\mathcal{H}^{tan} f(\vec{0})\|_F^2 > 0$. For reference, the variable *problem4_coords* is a data matrix containing samples from this manifold. The corresponding geodesic distances are saved in the variable *problem4_geods*. A scatterplot of the data (color coded by their geodesic distance to the origin) in shown below.

```
scatterplot3d(problem4_coords[,1],problem4_coords[,2],problem4_coords[,3],
              color = myColorRamp(c("blue","red","yellow"),problem4_geods),
              angle = 120,
              xlab = "", ylab = "", zlab = "", pch = '.', cex.symbols = 3)
```




```
# an interactive scatterplot of the data
# library(plotly)
# plot_ly(x=problem4_coords[,1],y=problem4_coords[,2],z=problem4_coords[,3],
#         marker = list(color =myColorRamp(c("blue","red","yellow"),problem4_geods),
#                       size = 3))
```

```
library(numDeriv)

Psi <- function(u) c(sign(pi - u[1]) * (1 + cos(u[1])), sin(u[1]), u[2])
F   <- function(x) sqrt(sum(x^2))  # the geodesic is isometric so we can use the euclidean distance

u0  <- c(0, 0)
x0  <- Psi(u0)

J   <- jacobian(Psi, u0)

Hx  <- hessian(F, x0)
gF  <- grad(F, x0)

Hpsi1 <- hessian(function(u) Psi(u)[1], u0)
Hpsi2 <- hessian(function(u) Psi(u)[2], u0)
Hpsi3 <- hessian(function(u) Psi(u)[3], u0)

Htan <- t(J) %*% Hx %*% J + gF[1]*Hpsi1 + gF[2]*Hpsi2 + gF[3]*Hpsi3

fro  <- sqrt(sum(Htan^2))
skew <- max(abs(Htan - t(Htan)))

print(list(Frobenius_norm=as.numeric(fro),
           Symmetry_violation=as.numeric(skew),
           H_tangent=Htan))
```

```
## $Frobenius_norm
```

```
## [1] 0.7071074
##
## $Symmetry_violation
## [1] 0
##
## $H_tangent
##                [,1]           [,2]
## [1,] -4.999999e-01 -3.368656e-07
## [2,] -3.368656e-07  5.000011e-01
```

the frobenius norm is positive for our speciifcation and is symmetric.

Problem 5

Let $\mathbf{L} = \mathbf{D} - \mathbf{A}$ be the graph Laplacian. For data matrix $\mathbf{Y} \in \mathbb{R}^{N \times t}$ with rows $\vec{y}_1^T, \ldots, \vec{y}_N^T$, verify the identity

$$tr(\mathbf{Y}^T \mathbf{L} \mathbf{Y}) = \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \mathbf{A}_{ij} \|\vec{y}_i - \vec{y}_j\|^2.$$

You may find the cyclic property of trace, $tr(\mathbf{ABC}) = tr(\mathbf{BCA}) = tr(\mathbf{CAB})$, to be helpful.

Claim:

$$\operatorname{tr}(Y^\top L Y) = \tfrac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N} A_{ij}\,\|y_i - y_j\|^2$$

with

$$L = D - A, \quad D_{ii} = \sum_j A_{ij}.$$

Start from the RHS and expand the squared distance:

$$\|y_i - y_j\|^2 = y_i^\top y_i + y_j^\top y_j - 2\,y_i^\top y_j.$$

Multiply by

$$A_{ij}$$

, sum over

$$i, j$$

, and use symmetry

$$A_{ij} = A_{ji}$$

so the first two terms contribute equally:

$$\frac{1}{2}\sum_{i,j} A_{ij}\|y_i - y_j\|^2 = \frac{1}{2}\sum_{i,j} A_{ij}(y_i^\top y_i + y_j^\top y_j) - \sum_{i,j} A_{ij} y_i^\top y_j = \sum_i \Big(\sum_j A_{ij}\Big) y_i^\top y_i - \sum_{i,j} A_{ij} y_i^\top y_j.$$

Let

$$d_i = \sum_j A_{ij}$$

and

$$D = \operatorname{diag}(d_1, \ldots, d_N)$$

. Recognize the trace forms:

$$\sum_i d_i\, y_i^\top y_i = \operatorname{tr}(Y^\top D Y),$$

since

$$\operatorname{tr}(Y^\top D Y) = \sum_i d_i\,\|y_i\|^2.$$

For the cross term, use the cyclic property

$$\operatorname{tr}(ABC) = \operatorname{tr}(BCA)$$

:

$$\operatorname{tr}(Y^\top A Y) = \operatorname{tr}(A Y Y^\top) = \sum_{i,j} A_{ij}\, y_i^\top y_j \quad \Rightarrow \quad \sum_{i,j} A_{ij}\, y_i^\top y_j = \operatorname{tr}(Y^\top A Y).$$

Substitute:

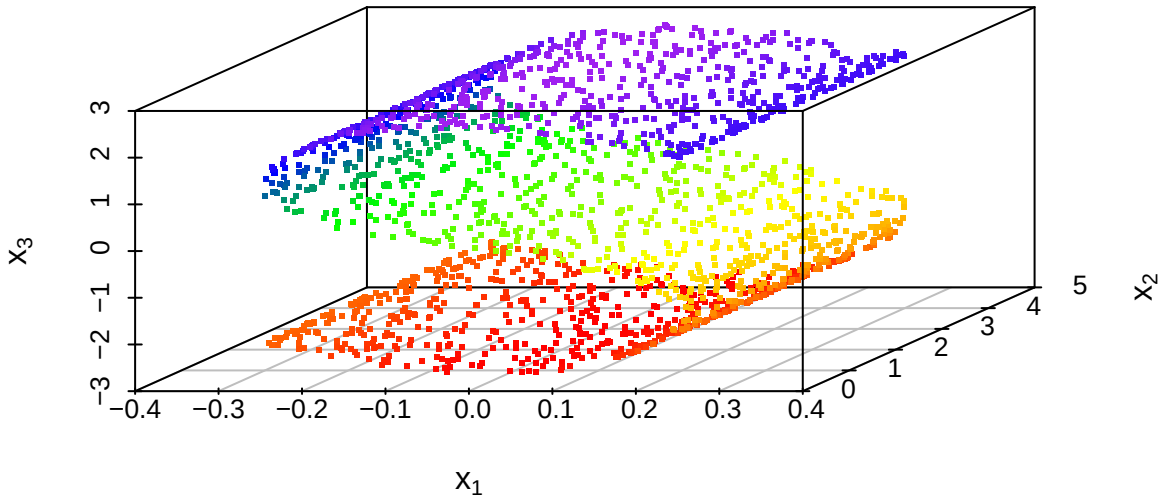
$$\frac{1}{2}\sum_{i,j} A_{ij}\|y_i - y_j\|^2 = \operatorname{tr}(Y^\top D Y) - \operatorname{tr}(Y^\top A Y) = \operatorname{tr}\big(Y^\top (D - A) Y\big) = \operatorname{tr}(Y^\top L Y).$$

Problem 6

In this problem, we will investigate the effect of noise on the performance of ISOMAP, HLLE, and Laplacian Eigenmaps. The variable *S$points* contains 2000 samples from a S-shaped sheet in $\mathbb{R}^3$. To assist with plotting, associated colors for each point are stored in the variable *S$colors*. For reference, the manifold is shown

below..

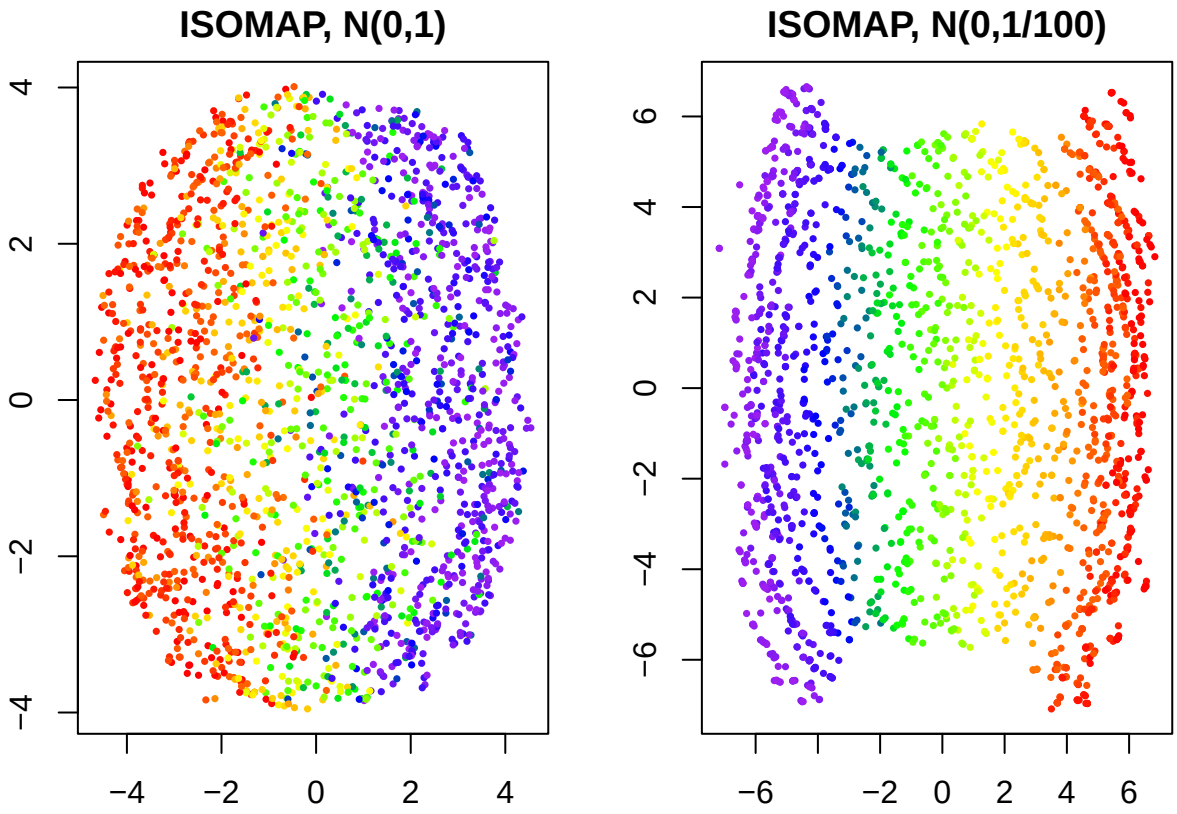
```
scatterplot3d(S$points,
              color = S$colors,
              xlab = expression(x[1]), ylab = expression(x[2]), zlab = expression(x[3]),
              pch = '.',cex.symbols = 3)
```



a) Add independent $N(0,1)$ noise to each entry of *S$points*. Generate a two-dimensional representation of the data using ISOMAP with $k = 30$ nearest neighbors. Repeat with $N(0, 1/100)$ noise.

```
X <- S$points; cols <- S$colors
iso_run <- function(s) do.isomap(X + matrix(rnorm(length(X), 0, s), nrow=nrow(X)), ndim=2, type=c("knn"
Y1 <- iso_run(1)
Y01 <- iso_run(0.1)

par(mfrow=c(1,2), mar=c(3,3,2,1))
plot(Y1, col=cols, pch=16, cex=.5, xlab="", ylab="", main="ISOMAP, N(0,1)")
plot(Y01, col=cols, pch=16, cex=.5, xlab="", ylab="", main="ISOMAP, N(0,1/100)")
```



b) Repeat part a using HLLEs and Laplacian Eigenmaps. For both methods, use $k = 30$ nearest neighbors. For Laplacian Eigenmaps, use kernelscale equal to 1.

```
run_all <- function(s) {
  Xn <- X + matrix(rnorm(length(X), 0, s), nrow=nrow(X))
  hlle <- as.matrix(embed(dimRedData(as.data.frame(Xn)), "HLLE", knn=30, ndim=2)@data@data[,1:2])
  le   <- do.lapeig(Xn, ndim=2, type=c("knn",30), kernelscale=1)$Y
  list(hlle=hlle, le=le)
}

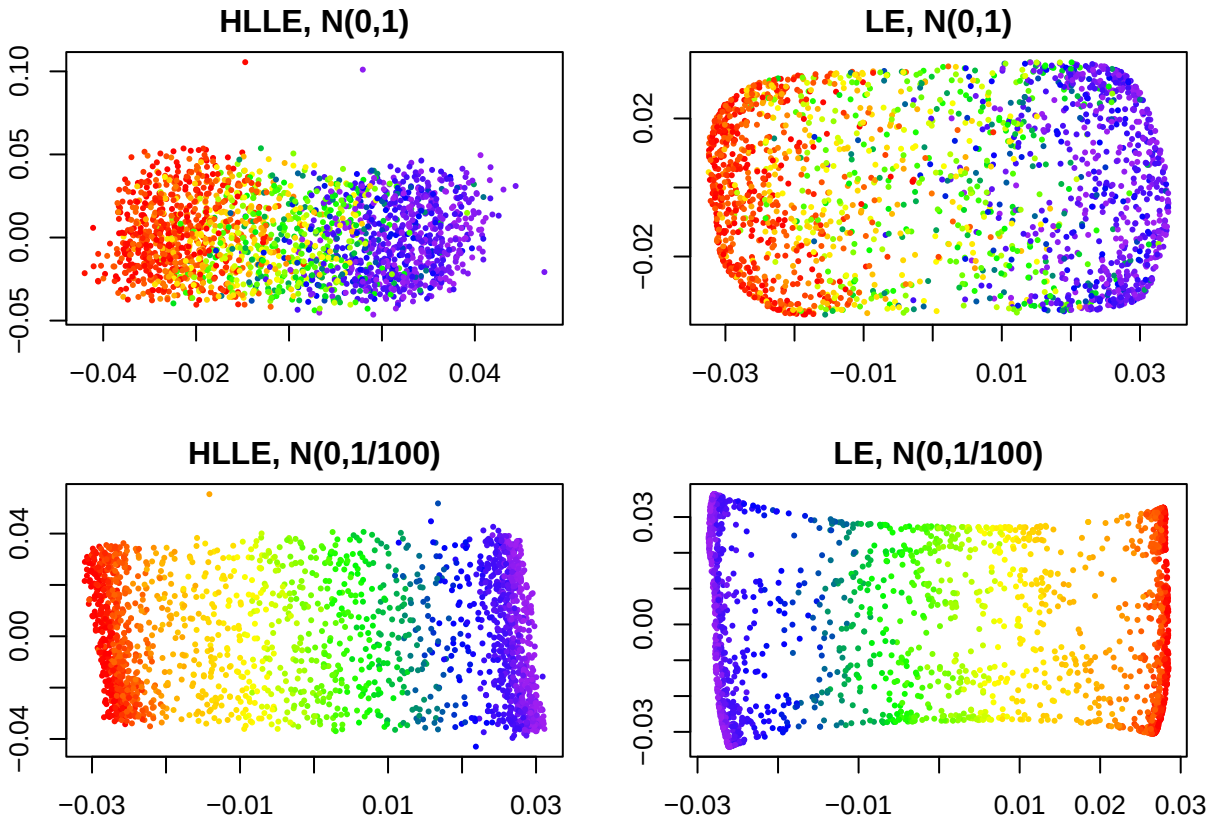
R1  <- run_all(1)
```

```
## 1/20002/20003/20004/20005/20006/20007/20008/20009/200010/200011/200012/200013/200014/200015/200016/2
```

```
R01 <- run_all(0.1)
```

```
## 1/20002/20003/20004/20005/20006/20007/20008/20009/200010/200011/200012/200013/200014/200015/200016/2
```

```
par(mfrow=c(2,2), mar=c(3,3,2,1))
plot(R1$hlle,  col=cols, pch=16, cex=.5, xlab="", ylab="", main="HLLE, N(0,1)")
plot(R1$le,    col=cols, pch=16, cex=.5, xlab="", ylab="", main="LE, N(0,1)")
plot(R01$hlle, col=cols, pch=16, cex=.5, xlab="", ylab="", main="HLLE, N(0,1/100)")
plot(R01$le,   col=cols, pch=16, cex=.5, xlab="", ylab="", main="LE, N(0,1/100)")
```


HLLE, N(0,1)
LE, N(0,1)
HLLE, N(0,1/100)
LE, N(0,1/100)
0.10
0.05
0.00
−0.05
−0.04
−0.02
0.00
0.02
0.04
0.02
−0.02
−0.03
−0.01
0.01
0.03
0.04
0.00
−0.04
−0.03
−0.01
0.01
0.03
0.03
0.00
−0.03
−0.03
−0.01
0.01
0.02
0.03